

TEST TECHNIQUE — SOUTENANCE

ALTEN SHOP

Application e-commerce full-stack · Angular + Spring Boot

Angular 18

Spring Boot 4 (Java 21)

JWT · BCrypt

Tests JUnit + Jest

Docker

Déployé sur Render

Présenté par : _____ · Juin 2026

1. Contexte & objectif

Enrichir le site e-commerce d'ALTEN avec de nouvelles fonctionnalités, côté **front ET back** (profil full-stack).

Ce qui était demandé

- **Front** : affichage produits, panier (ajout/retrait/badge/page), page Contact validée
- **Back** : API de gestion de produits, connexion JWT, restriction admin, panier & wishlist
- **Bonus** : pagination/filtrage, ajustement quantités, tests/Swagger

Le point de départ

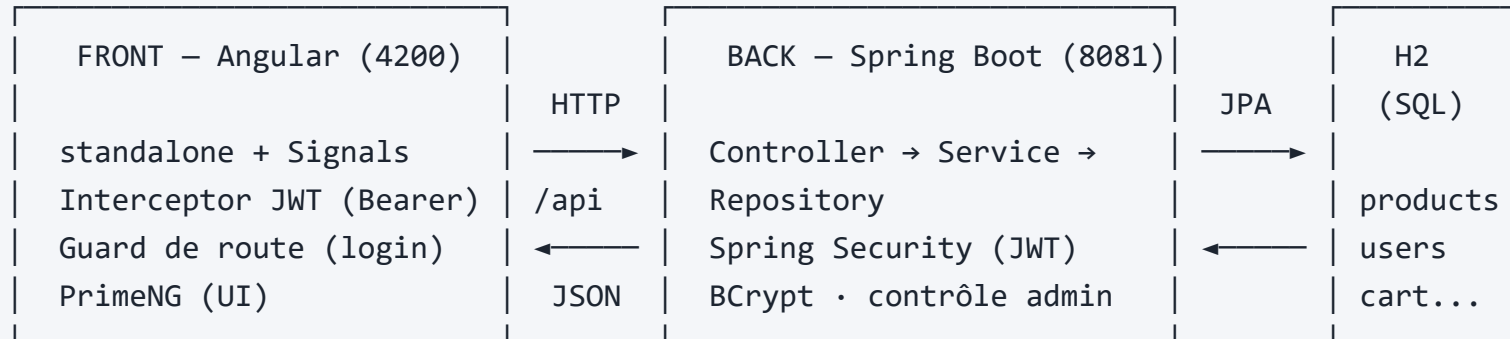
- Un front Angular **partiel** fourni (liste produits minimale, CRUD en mémoire)
- Un dossier back **vide** → tout à construire
- 30 produits de démonstration en JSON

Résultat 100 % du sujet couvert + tous les bonus, en ligne.

2. Stack technique & justification des choix

Brique	Choix	Pourquoi
Front	Angular 18 + PrimeNG, Signals	Base fournie ; Signals = état réactif moderne sans la lourdeur de NgRx
Back	Java 21 + Spring Boot 4	Standard d'entreprise pour les API REST, écosystème mature, sécurité éprouvée
Persistence	Spring Data JPA + H2	CRUD et schéma générés ; H2 = zéro installation ; bascule PostgreSQL en 1 ligne de config
Sécurité	Spring Security + JWT + BCrypt	Authentification stateless, mots de passe jamais en clair
Tests	JUnit 5 / Mockito (back) · Jest (front)	Tests écrits au fur et à mesure ; Jest choisi car Karma est déprécié
Doc API	springdoc-openapi (Swagger)	Documentation interactive et testable, générée automatiquement
Industrialisation	Docker + Render	Image portable, déploiement reproductible, démo en ligne

3. Architecture



Front en couches

Découpage par fonctionnalité : `data-access` (services + modèles), `features` (pages), `ui` (composants). État réactif par Signals.

Back en couches

`Controller` (routes REST) → `Service` (métier) → `Repository` (base). Sécurité transversale par filtre JWT, API stateless.

4. Fonctionnalités — Front

Partie 1 — Shop fait

- Liste produits riche : image, prix (€), note, **badge de stock coloré**
- Ajout au panier (+ notification) ; retrait
- **Badge** de quantité dans la barre du haut
- Page panier : quantités ajustables (+/-), sous-totaux, total
- Panier **persistant** (localStorage) → survit au rechargement

Partie 2 — Contact fait

- Menu « Contact » + formulaire réactif validé
- Email & message obligatoires, message ≤ 300 (compteur)
- Bouton désactivé si invalide, message de succès

Bonus fait

- Pagination de la liste
- Filtrage par nom + par catégorie
- Ajustement des quantités (liste & panier)

Intégration & sécurité côté front

- Écran de **connexion / inscription**
- **Interceptor** qui ajoute le token à chaque requête
- **Guard** : connexion obligatoire pour accéder à l'app
- Boutons de gestion produits **visibles uniquement par l'admin**

5. Fonctionnalités — Back & sécurité

API produits fait

- `GET /api/products` (+ par id), `POST`, `PATCH`, `DELETE`
- Validation des entrées, codes HTTP corrects (201/400/404/204)
- 30 produits + compte admin chargés au démarrage

Panier & wishlist fait

- `/cart` et `/wishlist` par utilisateur
- Utilisateur déduit du token → **isolation** (on ne voit que le sien)

Authentification JWT fait

- `POST /account` (inscription), `POST /token` (connexion)
- Mots de passe hachés **BCrypt** (jamais en clair)
- Connexion obligatoire pour accéder à l'API

Contrôle admin fait

- Seul `admin@admin.com` peut créer/modifier/supprimer un produit
- Filtre JWT → rôle déduit de l'email → `403` sinon

Bonus Swagger UI + tests automatisés.

6. Qualité & industrialisation

Tests automatisés (au fur et à mesure)

Back : JUnit 5 + Mockito + MockMvc — CRUD, validation, JWT, contrôle d'accès (401/403/admin), panier, wishlist.

Front : Jest — service d'auth, service panier (dont persistance), formulaire contact.

Documentation

Swagger interactif (bouton « Authorize » pour tester les routes protégées) + journal de bord détaillé.

Conteneurisation

`Dockerfile` multi-étapes : compilation dans une image JDK, exécution dans une image JRE légère → image finale optimisée.

Déploiement continu

Code sur GitHub → Render redéploie automatiquement à chaque push. Back (Docker) + Front (site statique) en ligne, accessibles par lien.

CORS & environnements

Proxy en dev, CORS + URL d'API par environnement en prod.

7. Démonstration — parcours proposé

1. Ouvrir `alten-shop.onrender.com` → redirection automatique vers **/login** (connexion imposée).
2. Se connecter en **admin** (`admin@admin.com`) → la liste des produits du back s'affiche.
3. Montrer la **recherche / le filtre par catégorie** et la **pagination**.
4. Ajouter des produits au **panier** → le **badge** s'incrémente ; ouvrir le panier, ajuster les quantités.
5. **Rafraîchir la page** → le panier est toujours là (persistance).
6. En admin : **créer / modifier / supprimer** un produit (boutons visibles).
7. Se déconnecter, créer un **compte simple** → les boutons de gestion **disparaissent** (sécurité).
8. Page **Contact** : validation du formulaire + message de succès.
9. Montrer **Swagger** : tester une route protégée avec le token.

Astuce : ouvrir le lien ~1 min avant pour « réveiller » le serveur gratuit.

8. Difficultés rencontrées & solutions

Problème	Solution apportée
Spring Boot 4 utilise Jackson 3 (nouveau namespace)	Adaptation des imports — code aligné sur la toute dernière version
Karma (tests Angular) est déprécié et absent du projet	Migration vers Jest (moderne, rapide, sans navigateur)
Un refus <code>403</code> était transformé à tort en <code>401</code>	Cause : re-contrôle de la page d'erreur interne ; exclusion des dispatch <code>ERROR</code> / <code>FORWARD</code>
Render impose le port via <code>\$PORT</code>	Port rendu dynamique : <code>server.port=\${PORT:8081}</code>
Le panier disparaissait au rechargement	Persistance dans le <code>localStorage</code>
Un non-admin voyait les boutons de gestion (faux succès)	Boutons masqués + remontée réelle des erreurs serveur (toast)

→ Chaque problème a été **diagnostiqué puis vérifié en conditions réelles**, pas seulement en test.

9. Bilan & pistes d'amélioration

Couverture du sujet

Exigence	
Front Partie 1 (Shop)	✓
Front Partie 2 (Contact)	✓
Back Partie 1 (CRUD)	✓
Back Partie 2 (JWT, admin, panier, wishlist)	✓
Bonus (pagination, quantités, Swagger, tests)	✓

Pistes d'amélioration

- Brancher le panier front sur l'API panier du back (persistance multi-appareils)
- Externaliser le secret JWT et le mot de passe admin en variables d'environnement
- Base PostgreSQL persistante (ex : via le profil prod)
- Pipeline CI : exécuter les tests automatiquement à chaque push
- Pagination/filtrage côté serveur pour de gros volumes

MERCI DE VOTRE ATTENTION

Questions ?

Démo : alten-shop.onrender.com

Code : github.com/anas34b/alten-shop

admin@admin.com / admin123